

GMARK ERP

Complete Project Documentation

Easy-to-understand guide for interns, developers, testers and new team members

How to use this guide

If you are completely new, read Sections 1–6 first. Then follow Section 8 to install the project and Section 11 to run it. Use the remaining sections as a reference while working.

Important: repository URLs, passwords, cloud credentials and production values must be obtained from the project owner. Never guess or share secrets.

1. Project Introduction

Gmark ERP (also called WeighPro ERP) is a business management application for agricultural trading, retail, manufacturing and industrial procurement. It connects customers, suppliers, products, purchases, sales and financial transactions in one system.

The most important thing for a new team member to understand is that ERP is a connected business process. A change in one document can affect another document, quantity, tax, ledger or settlement.

Example

Supplier → Purchase Order → GRN → Purchase Invoice → Payment → Ledger

2. What is an ERP?

ERP means Enterprise Resource Planning. In simple words, it is software that helps a business manage its day-to-day operations in one connected application.

```
Customer / Supplier Master
↓
Product / HSN / Unit Master
↓
Sales or Purchase Transaction
↓
Invoice / Voucher
↓
Ledger / Settlement
```

When working on any feature, always ask: **What other feature depends on this data?**

3. What Gmark ERP Does

Area	Purpose
Account Management	Customers, suppliers and other business parties.
Product Management	Products, units, categories and tax-related information.
Sales	Sales Orders, Delivery Challans and Sales Invoices.
Purchase	Purchase Orders, GRNs and Purchase Invoices.
Finance	Receipts, Payments, Journals, Contra entries and ledger transactions.
Tax	GST calculation according to transaction rules.
Import / Export	Bulk Excel import and data export.
Documents	Business document uploads.
Users / Security	Login, tokens, roles and permissions.
Languages	English, Marathi and Hindi.

4. Complete Business Flow

Learn these flows before reading the code.

Sales Flow

```
Customer
↓
Sales Order (SO)
↓
Sales Delivery Challan
↓
Sales Invoice (SI)
```

↓
Receipt / Settlement
↓
Customer Ledger

Purchase Flow

Supplier
↓
Purchase Order (PO)
↓
GRN
↓
Purchase Invoice (PI)
↓
Payment / Settlement
↓
Supplier Ledger

Finance Flow

Business Transaction
↓
Receipt / Payment / Journal / Contra
↓
Debit / Credit
↓
Ledger
↓
Settlement

Why the flow matters

If Purchase Order quantity changes, the GRN and Purchase Invoice may depend on that quantity. Therefore, test the complete affected flow, not only one screen.

5. Users and Roles

Role	Easy explanation
Superadmin	Highest-level administrative and system control.
Administrator	Manages business operations, masters and finance.
Seller	Works mainly with sales, customers and billing.
Buyer	Works mainly with suppliers, procurement, POs and GRNs.
Operator	Performs routine data entry and voucher operations.

For an access problem, check authentication (logged in?) and authorization (role allowed?).

6. Technology Used

Technology	Simple meaning	Use in project
React 19	Frontend library	Screens, forms, tables and UI
Vite 7	Frontend build tool	Runs and builds React
Redux Toolkit / Saga	State management	Shared state and async/API work
NestJS 11	Backend framework	APIs and business logic
TypeScript 5.9	Typed JavaScript	Safer application code
Prisma 6.19	Database ORM	Communicates with PostgreSQL

Technology	Simple meaning	Use in project
PostgreSQL 14+	Database	Stores permanent ERP data
Redis / BullMQ	Cache/queue	Background processing where configured
Swagger	API documentation	Explore and test APIs
i18next	Translation	English, Marathi, Hindi
AWS SNS	SMS service	OTP in normal mode

Overall architecture

```

Browser
↓
React Frontend
↓ HTTP / REST + JWT
NestJS Backend
↓
Service / Business Logic
↓
Prisma
↓
PostgreSQL

```

7. Project Structure

There are two main applications: frontend and backend.

Frontend

```

frontend/
  ■■■■ src/
  ■ ■■■■ assets/ → static assets
  ■ ■■■■ components/ → reusable UI
  ■ ■■■■ constants/ → shared constants
  ■ ■■■■ i18n/ → translations
  ■ ■■■■ pages/ → screens
  ■ ■■■■ redux/ → state management
  ■ ■■■■ utils/ → helpers
  ■■■■ .env
  ■■■■ package.json

```

Backend

```

backend/
  ■■■■ src/
  ■ ■■■■ common/ → shared utilities
  ■ ■■■■ modules/
  ■ ■ ■■■■ Admin/
  ■ ■ ■■■■ Auth/
  ■ ■ ■■■■ Ledger/
  ■ ■ ■■■■ Master/
  ■ ■ ■■■■ Purchase/
  ■ ■ ■■■■ Sales/
  ■ ■ ■■■■ User/
  ■ ■■■■ prisma/
  ■ ■ ■■■■ schema.prisma
  ■ ■ ■■■■ seed.ts
  ■ ■ ■■■■ seed-hsn.ts
  ■ ■ ■■■■ seed-pincodes.ts
  ■ ■■■■ app.module.ts
  ■ ■■■■ main.ts
  ■■■■ uploads/
  ■■■■ .env
  ■■■■ package.json

```

For a feature, trace: **Frontend page → API → Controller → Service → Prisma → Database**.

8. Requirements and Installation

Software	Requirement
OS	Windows 10/11, macOS 12+ or Ubuntu 20.04/22.04 LTS
Node.js	18.18.0 or 20.x LTS; recommended 20.11.0
npm	9.x or 10.x
PostgreSQL	14.x, 15.x or 16.x
Redis	6.x or 7.x when required
Browser	Latest Chrome, Edge, Firefox or Safari
RAM	Minimum 8 GB; 16 GB recommended
Free storage	At least 10 GB

Step 1 — Install Git, Node.js, PostgreSQL and VS Code.

Step 2 — Verify Node.js

```
node --version
npm --version
```

Step 3 — Clone the repository

```
git clone <repository-url>
cd <project-folder>
```

Use the repository URL supplied by your project owner.

9. Environment Configuration

Environment files contain machine-specific settings. Never commit real passwords or cloud credentials.

Backend: backend/.env

Variable	Purpose
PORT	Backend port; documented local value is 3001.
DATABASE_URL	PostgreSQL connection string.
JWT_SECRET	Access-token signing secret.
JWT_EXPIRATION	Access-token expiry.
JWT_REFRESH_SECRET	Refresh-token signing secret.
JWT_REFRESH_EXPIRATION	Refresh-token expiry.
SWAGGER_USER / SWAGGER_PASSWORD	Swagger protection.
AWS_REGION	AWS region.
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY	AWS credentials; obtain securely.
ENABLE_DEV_OTP / DEV_OTP	Development OTP mode.

Frontend: frontend/.env

Variable	Purpose
VITE_API_BASE_URL	API base URL, e.g. http://localhost:3001/api/v1
VITE_SERVER_URL	Backend URL, e.g. http://localhost:3001

Never commit .env files containing secrets.

10. Database and Prisma

PostgreSQL is the database. Prisma is the tool used by NestJS to communicate with PostgreSQL.

React → NestJS → Prisma → PostgreSQL

Term	Easy meaning
Database	Container holding application data.
Table	Stores one type of information.
Row	One record.
Column	One field.
Primary Key	Unique record ID.
Foreign Key	Connects records.
Relation	Describes table connections.

Prisma schema:

```
backend/src/prisma/schema.prisma
```

Database setup:

```
cd backend
npx prisma generate
npx prisma migrate dev
```

Seed data:

```
npm run seed
npm run seed:hsn
npm run seed:pincodes
```

11. How to Run the Project

Normal local-development process:

1. Start PostgreSQL.

2. Start backend in Terminal 1.

```
cd backend
npm install
npm run start:dev
```

Backend:

```
http://localhost:3001
```

Swagger:

```
http://localhost:3001/api/docs
```

3. Start frontend in Terminal 2.

```
cd frontend
npm install
npm run dev
```

Frontend:

```
http://localhost:5173
```

4. Open the frontend in the browser.

If the frontend opens but data does not load, check backend status and VITE_API_BASE_URL.

12. Authentication and Security

The project uses phone OTP authentication. Successful verification provides access and refresh tokens.

```
Phone Number
↓
Request OTP
↓
Verify OTP
↓
Access + Refresh Token
↓
Authenticated API Requests
```

Developer OTP

ENABLE_DEV_OTP and DEV_OTP can be used during development so an actual SMS is not required. Normal operation can use AWS SNS.

JWT

JWT identifies the logged-in user. API requests normally send an Authorization Bearer token.

Security rules

- Do not share passwords, JWT secrets or AWS credentials.
- Do not commit secrets to Git.
- Do not bypass authentication to make a feature work.
- Check roles and guards for permission problems.

13. Master Data

Master data is information created once and reused by transactions.

Master	Purpose
Account Master	Customers, suppliers and business parties.
Product Master	Products and product details.
Unit Master	Measurement/quantity units.
Group Master	Account classification.
Category Master	Product classification.
HSN/SAC Master	Tax classification codes and rates.
Pincode/Location Data	Address and location selection.

Example

```
Create Customer
↓
Account Master
↓
Customer appears in Sales Order
↓
Customer appears in Sales Invoice
↓
Customer transactions affect Ledger
```

If a transaction dropdown is empty, first check the required master record.

14. Sales Process

```
Customer
↓
Sales Order (SO)
↓
Delivery Challan
↓
Sales Invoice
↓
Receipt / Settlement
```

Sales Order records what the customer wants to buy, including customer, product, quantity, price, discount and tax information.

Delivery Challan records delivery of goods.

Sales Invoice records the billable transaction, tax and total.

Development rule: if Sales Order quantity or pricing changes, verify related Challan and Invoice behavior.

15. Purchase Process

```
Supplier
↓
Purchase Order (PO)
↓
GRN
↓
Purchase Invoice
↓
Payment / Settlement
```

Purchase Order records what the company intends to purchase.

GRN records goods actually received and is important for quantity tracking.

Purchase Invoice records the supplier bill, taxes, expenses and total.

Quantity example

```
PO = 10 units
GRN = 5 units
Remaining = 5 units
```

Always test quantity changes through the complete purchase flow.

16. Finance and Ledger

Finance records money-related transactions and provides the ledger history of accounts.

Transaction	Purpose
Receipt	Money received.
Payment	Money paid.
Journal	Accounting adjustment/entry.
Contra	Transfer between cash/bank-type accounts.
Settlement	Connects receipt/payment to outstanding bills.
Ledger	Financial history of an account.

```
Voucher
↓
```



```

Debit / Credit
↓
Ledger
↓
Outstanding Balance
↓
Settlement

```

17. GST and Important Business Rules

The system uses state-based GST logic.

Transaction type	Tax treatment
Same state / intra-state	GST is split into CGST + SGST.
Different state / inter-state	GST is applied as IGST.

Example: 18% GST within the same state becomes CGST 9% + SGST 9%. Inter-state becomes IGST 18%.

Do not duplicate business calculations in multiple frontend screens. Check existing backend/service logic before adding a new calculation.

18. API and Frontend/Backend Communication

```

User clicks Save
↓
React sends HTTP request
↓
NestJS Controller
↓
DTO / Validation / Guard
↓
Service
↓
Prisma
↓
PostgreSQL
↓
Response to React

```

Swagger

With backend running, open:

<http://localhost:3001/api/docs>

Area	Examples
Authentication	OTP, verification, refresh token, logout
Sales	Orders, challans, invoices
Purchase	Orders, GRN, invoices
Masters	Accounts, products, groups, HSN, units
Ledger	Transactions, vouchers and settlements

When an API fails, inspect the browser Network tab first and then backend logs.

19. Import, Export and File Management

The project includes a procurement bulk-import processor for structured Excel import of Purchase Orders, GRNs and Purchase Invoices. It handles supplier/product mapping and quantity, rate and tax validation.

Excel export is available from table pages using the configured `xlsx/xlsx-js-style` functionality.

Uploaded business documents are stored under the backend uploads area. Examples include MSME certificates, customer PO copies and supplier invoice scans.

Do not change or delete file-storage logic without checking where files are referenced.

20. Language Support

The project supports English (en), Marathi (mr) and Hindi (hi) through i18next.

```
Language selected
↓
i18next locale
↓
Translation key
↓
Translated UI
```

Translation files:

```
frontend/src/i18n
```

When adding a new label, use the existing translation-key approach instead of hardcoding the same text in many components.

21. How to Debug a Problem

```
1. Understand requirement
↓
2. Reproduce issue
↓
3. Browser Console
↓
4. Network request
↓
5. Backend terminal
↓
6. Controller
↓
7. Service
↓
8. Prisma / Database
↓
9. Fix root cause
↓
10. Test again
```

Console: frontend errors. Network: API URL, method, payload, status and response. Backend terminal: validation, business logic and database errors.

Do not immediately change code. First identify where the request is failing.

22. Common Errors

Problem	Likely reason	First check
Port 3001 in use	Old backend process	Stop old development process.
Port 5173 in use	Old Vite process	Stop old development process.
P1001 / DB error	PostgreSQL or URL issue	PostgreSQL + DATABASE_URL
401	Token missing/invalid/expired	Login + Authorization header
403	Role lacks permission	Role + guards
404	Wrong endpoint/route	API URL + backend route
500	Backend exception	Backend logs

Problem	Likely reason	First check
Empty dropdown	Master missing	Relevant master
Translation key visible	Missing translation	frontend/src/i18n
Frontend cannot load data	API/backend issue	VITE_API_BASE_URL + backend

Windows PowerShell port check:

```
Get-NetTCPConnection -LocalPort 3001
Get-NetTCPConnection -LocalPort 5173
```

23. Git and Development Workflow

Use a feature branch for each task.

```
git pull origin main
git status
git switch -c feature/my-change

# make changes
# test application

git add .
git commit -m "Describe the change"
git push -u origin feature/my-change
```

Before pushing

- Test the changed feature.
- Test related flows.
- Check browser console and backend logs.
- Review git diff.
- Make sure no secrets are included.
- Keep the commit focused.

Avoid git push --force, git reset --hard or rebase unless a senior developer specifically instructs you.

24. Testing Checklist

Check	Question
Requirement	Did I implement exactly what was requested?
UI	Does the screen display correctly?
Validation	Are invalid inputs rejected properly?
API	Does the correct endpoint receive correct data?
Database	Is expected data saved/updated?
Success	Does the normal flow work?
Failure	Does the system show a useful error?
Related flow	Could another document/module be affected?
Permissions	Does the correct role have access?
Logs	Are there unexpected errors?
Git	Is the change clean and focused?

Example: after changing Sales Order quantity logic, test Sales Order, related Challan and related Invoice behavior.

25. Production Overview

Interns should not deploy to production without approval.

```
Developer Code
↓
Git Repository
↓
Build
↓
AWS / Server
↓
NestJS + PostgreSQL
↓
Nginx
↓
User Browser
```

Build commands

```
cd backend
npm run build

cd frontend
npm run build
```

The frontend build is generated under frontend/dist. Backend runs using the approved server/process-manager setup. Nginx can serve the frontend and proxy API requests.

Production safety

- Use production environment values.
- Never expose database passwords or JWT secrets.
- Do not run destructive database commands without approval.
- Follow backup and deployment procedures.
- Test before deployment.

26. Important Commands Cheat Sheet

Task	Command
Install dependencies	npm install
Start backend	npm run start:dev
Start frontend	npm run dev
Generate Prisma client	npx prisma generate
Development migration	npx prisma migrate dev
Seed base data	npm run seed
Seed HSN	npm run seed:hsn
Seed pincodes	npm run seed:pincodes
Build backend	npm run build
Build frontend	npm run build
Swagger	http://localhost:3001/api/docs

27. Intern Learning Plan

Stage	Learn
Day 1	ERP concept, project purpose, install tools and run project.
Days 2–3	React pages/components and NestJS modules/controllers/services.
Days 4–5	PostgreSQL, Prisma, tables, relationships and Swagger.
Week 2	Masters, Sales, Purchase, GST and Ledger.
Week 3	Debugging, validation, import/export and translations.
Week 4	Small UI/API changes, testing, Git and pull requests.

Do not try to memorize the whole codebase. Learn how a requirement moves from UI → API → business logic → database and back.

28. Final Checklist

I can...	Done
Explain what Gmark ERP is.	■
Explain why master data is important.	■
Explain Sales: SO → Challan → Invoice.	■
Explain Purchase: PO → GRN → Invoice.	■
Explain Receipt, Payment and Ledger.	■
Explain CGST/SGST vs IGST.	■
Find a frontend page for a feature.	■
Find the related backend module/service.	■
Find the Prisma schema.	■
Run PostgreSQL, backend and frontend.	■
Open Swagger and understand an API.	■
Debug using Console and Network.	■
Use a Git feature branch.	■
Test success and error cases.	■
Keep secrets and production data safe.	■

THE GOLDEN RULE

Understand the business flow first → understand the code → make the smallest safe change → test the complete affected flow.

Prepared as an easy-to-understand version of the supplied Gmark ERP / WeighPro project documentation.